

通用神经网络处理器的结构感知切图与分层调度模型

摘要

多核神经网络处理器的执行效率受计算依赖、共享带宽与核内存储共同限制。针对三种逐层增加同步和缓存能力的运行场景，本文在保持原图语义与附件调度规则的条件下，建立以完工时间为主、兼顾额外搬运的切图与调度方法。

首先，将张量 - 操作二部图投影为操作依赖图，提取弱连通分量、层宽、双管道工作量与共享输入。对于无核间缓存同步的场景，采用完整分量装箱与结构切分相结合的有限构造，以任务工作量和依赖估计筛选方案，并对优先候选进行有限重排与合并。具备硬件同步后，将通信边界转为核心边界，以张量生命周期估计核内驻留风险，在同核复用与计算均衡之间选择。对于共享只读 FIFO Cache，进一步建立 DDR、L2 双带宽池事件模型，描述发射查询、完成插入与淘汰，并用核内换出估计修正候选时间。

冻结算法分别从原始输入独立生成方案，再由附件程序验收。100 图五核平均加速比分别为 3.8311、3.9522 和 4.1939。第二问的平均额外搬运较第一问减少 36.17%；第三问固定同一方案开启 L2 后，平均耗时缩短 2.976%，平均字节命中率为 26.45%。三问已测正式方案均通过执行验收。独立新图检验同时发现第二问对输入表示和部分分层结构不够稳定，说明冷启动与有效的结构分析仍需配合独立性能验证。

本文通过有限结构构造和逐层细化的资源模型控制方案生成成本，用逐图对照区分通信减少、调度改变与缓存开关的效果。该方法在给定数据上兼顾求解速度与多核收益，后续改进重点为驻留时序近似和未见结构下的候选选择。

关键词：计算图切分；多核调度；张量生命周期；FIFO Cache；事件模型

第一章 问题重述

1.1 计算图在多核处理器上的执行任务

神经网络处理器的并行计算能力只有在数据及时到达、核内空间足够且任务依赖得到满足时才能发挥。矩阵与向量操作可以分配到不同核心，也可以在同一核心的不同管道上重叠执行；张量的搬入、搬出与临时驻留则占用另一组资源。神经网络加速器架构研究将计算组织和数据流组织视为密切相关的设计问题[1]。本题将这种耦合具体化为给定硬件配置下的计算图切分、核心分配与子图排序问题。

题目提供 100 个计算图以及固定的核内调度、多核模拟程序。原图由张量节点与操作节点构成，边表示数据依赖；张量大小、所在存储层、操作周期及计算管道均已给定。求解者需要将非搬运操作划入若干子图，再为各核心安排子图执行序列。输出接口只有操作到子图的映射和每核子图序列，实际指令时刻、缓存换入换出以及共享带宽分配由附件机制产生[2]。

切图同时改变了三个量：能够并行执行的计算工作量、跨任务或跨核心的数据传递量，以及张量在核内占用空间的时间。子图过大容易集中负载，子图过小又会增加边界搬运和同步。因而，问题的关键在于识别哪些依赖适合保留在核内，哪些计算区域值得以通信代价换取并行执行。

1.2 三种场景的具体要求

问题一采用场景 A：每个子图形成独立 Task，Task 之间不能直接延续核内缓存状态。即使两个子图位于同一核心，边界数据仍须经 DDR 传递；跨核依赖还具有固定等待开销。需要在这种机制下安排切图与任务顺序，尽量缩短系统完工时间并减少额外搬运。

问题二采用场景 B：同一核心的全部子图合并为一个 Task，同核生产与消费可以复用 L1/UB 中的数据。跨核依赖通过搬出、硬件同步和搬入建立联系。合并扩大了数据复用范围，也延长了部分张量的驻留，因此分核时需要兼顾通信内化、负载均衡与缓存压力。

问题三在场景 B 上增加所有核心共享的 1 MiB 只读 FIFO Cache。读取发射时查询缓存，命中读取使用独立的 L2 带宽；完成读取后按附件规则尝试插入，容量不足时按先入先出次序淘汰。需要利用合法的切图与排序形成共享读取机会，并比较同一方案在关闭和开启 L2 时的性能。

三问均要求在 1-5 核配置下汇总加速效果，保存逐图完工时间、额外搬运量及可复现方案。第三问还需给出 Cache 命中情况。问题一、二的单核参照采用整图单核方案；第三问对该固定方案补充缓存开关比较，不进行单核方案搜索。

第二章 问题分析与总体思路

2.1 并行收益为何受到数据流限制

依赖图中的独立分支提供并行执行机会，但计算量均衡并不等于完成时间均衡。一个核心即使承担较少计算，也可能长时间等待前驱数据；多个核心同时搬入时，又会竞争同一 DDR 带宽池。将一条大张量依赖切开，既增加搬运工作量，也可能把原本连续的计算链变成跨核等待链。判断一个切分是否值得，需要把计算、依赖与通信放在同一时间尺度上比较。

弱连通分量给出了较自然的起点：不同分量之间不存在计算依赖，保持分量完整可以避免切断其内部生产-消费关系。然而，分量可能共同读取同一个外部输入，分量内部也可能超过核内缓存容量。因此，完整分量装箱减少的是由内部切分引入的边界代价，并不意味着总 DDR 流量为

零。若最大分量占据大部分工作量，还必须考察其内部并行结构。

输入图的另一项差异来自计算管道。相同的操作周期总和可能对应不同的矩阵、向量工作量组成；若仅按总周期分核，两个核心可能分别拥堵在不同管道，或一个核心的单管道成为瓶颈。本文在分量和子图层面保留双管道负载，再通过通信与依赖估计比较可行划分。

2.2 从任务边界到数据驻留和读取时序

场景 A 中，切图边界直接决定 Task 边界，减少不必要边界有助于降低重复搬运。场景 B 中，同核子图可以连续复用数据，优化重点转向哪些操作应被安排在同一核心。把共享张量的消费者集中起来能够减少跨核读取，却也可能延迟张量的最后一次使用，使缓存长期被占用。本文以张量的首次出现和末次使用估计驻留区间，将容量压力引入候选排序。

只读 Cache 又增加了一项时间条件：第二次读取必须在首次读取完成之后、条目被淘汰之前发射，才能形成命中。共享输入很多并不足以说明收益大；如果各核同时发起首次读取，或共享张量很快被新条目挤出，潜在复用不会转化为带宽收益。第三问因而继承场景 B 的分核与驻留分析，并进一步展开读写请求的事件时序。

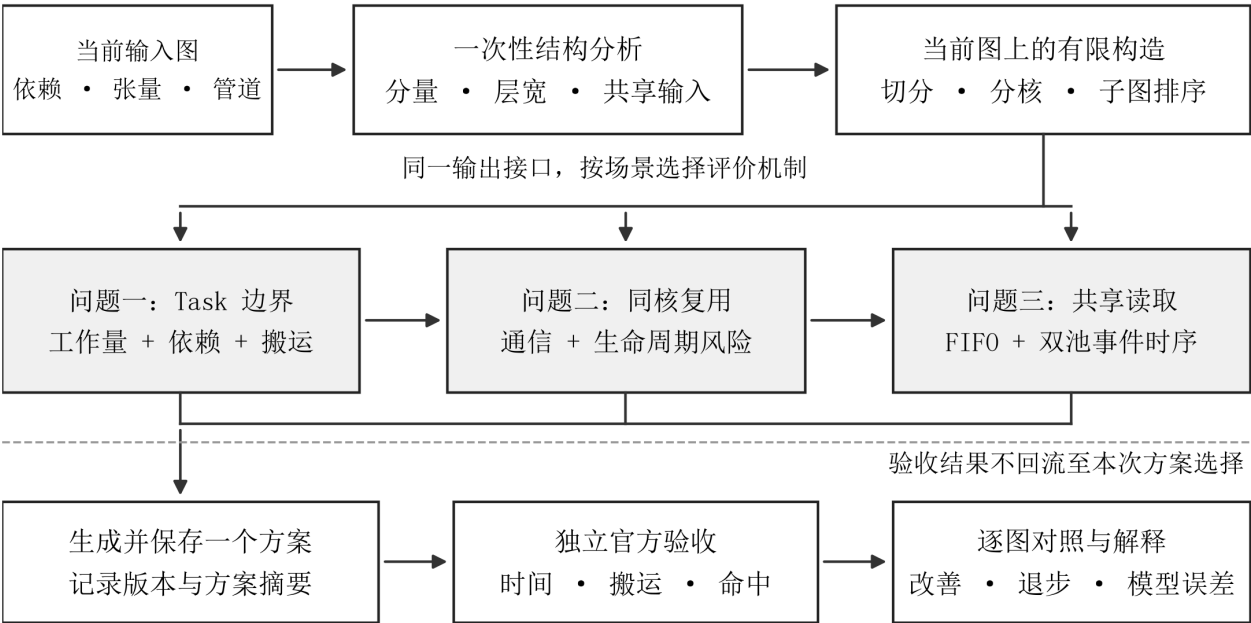


图 2-1 三问共用的决策过程及逐层增加的运行机制。输入结构在每个图上重新计算；实线连接求解所用信息，底部表示方案保存后的独立验收。图中节点为建模环节，不代表实际执行时间。

2.3 已有方法的启发与本题的实现范围

深度学习编译器研究强调图表示、调度和目标硬件之间的配合[3]。经典最长作业优先分配与列表调度分别提供了负载分散和关键路径优先的构造思路[4][5]。这些方法能够低成本产生初始方案，但本题的任务时间会随切图、缓存换出和共享带宽竞争变化，不能直接套用独立定长作业模型的性能界。

基于性能建模的调度研究说明，估计模型的用途在于把任务和资源信息转化为可比较的决策依据[6]。本文采用有限种结构构造，并让统一模型对当前图产生的方案进行排序。候选是一次求解过程中的中间决策，最终接口仍接收一个图并输出一个方案。构造数量受到显式限制，避免把大部分时间用于反复调用耗时的完整模拟。

MAGIS 将图优化与调度中的延迟、内存开销联合考虑[7]，Timeloop 用架构和数据流模型分析加速器的存储与执行代价[8]。本文从中吸收张量生命周期分析和资源服务量建模的思路，具体实现保持题目原计算图与固定硬件配置。既不进行算子融合、计算复制或重计算，也不引入题目未给出的张量分块与映射维度。

三个问题具有完工时间与额外搬运量两个评价方向。本文将完工时间置于主要位置，用搬运量和驻留风险修正时间代理或打破并列。当前算法采用确定性构造与评分选择，未执行非支配排序或种群进化；后文公式和伪代码均围绕这一实际求解过程展开。

第三章 模型假设与符号说明

3.1 题设条件与估计假设

张量大小、操作周期、数据依赖和存储位置以实际附件为准。计算图保持原语义，每个非 COPY 操作恰好执行一次；核心数、四条管道、缓存容量、同步延迟及带宽取题目配置。它们是约束模型的已知条件。

候选比较采用三项估计约定。第一，以当前图的计算工作量和依赖路径构造确定性的时间估计，不额外引入操作周期随机扰动。第二，问题一、二用任务或核心的汇总工作量近似资源拥堵，问题三将读写服务展开为事件；估计精度随模型层次增加，但不将任何一项代理量视为官方执行时间的恒等式。第三，核内驻留估计采用分析序列上的张量使用区间，实际空间复用及换出插入仍由固定执行机制确定。

这些约定使每次方案生成只依赖当前输入图、硬件配置和冻结参数。全局参数可以在开发阶段诊断、标定；正式生成方案时不读取历史答案或该图旧得分。已给定 100 图参与过开发，故全量成绩用于报告本题数据上的性能，独立输入测试另行列示。

3.2 统一符号与计量单位

原始图记为 $G = (T \cup O, E)$ ，非 COPY 操作集合为 O_c 。本文用 t 表示张量索引、 u, v 表示操作索引、 s 表示子图、 k 表示核心；物理时刻记为 θ 。张量大小统一用 byte，时间统一用 cycle，图中 MiB 按 2^{20} byte 换算。

表 3-1 主要符号及其含义，局部辅助量在对应公式前定义

符号	定义与单位
$s_t, c_u, p(u)$	张量字节数、操作周期、操作所属计算管道
$G_c = (O_c, E_c)$	由原图生产 - 消费关系投影得到的操作依赖图
C, C_j, K	弱连通分量集合、第 j 个分量及分量数
d_u, h, m_ℓ	操作拓扑深度、最大深度及第 ℓ 层操作数
N, k	核心数与核心编号, $k = 0, \dots, N-1$
S, ϕ, a, π_k	子图集合、操作归属、核心分配及核心 k 的子图序列
P	完整方案 $(\phi, a, \pi_0, \dots, \pi_{N-1})$
W_j^q, L_k^q	分量或核心在管道 $q \in \{M, V\}$ 上的工作量, cycle
C_r, β, β_{L2}	存储容量 byte; DDR、L2 总带宽 byte/cycle
$T^X(P), D^X(P)$	场景 X 中实际完工时间、官方额外搬运量
$\hat{T}, \hat{B}, J$	时间估计、字节数估计及候选评分
$T_{i,1}, \bar{S}_N$	第 i 图的整图单核参照、 N 核逐图加速比均值

带帽变量来自求解模型，无帽的执行时间和搬运结果来自方案冻结后的验收。 ϕ 与 π_k 对应提交字段； a 可由子图在核心序列中的位置恢复。该区分使建模解释、求解输出与结果统计保持一致。

第四章 统一决策模型与执行约束

4.1 切图、分核和排序的可行域

每个方案将 O_c 划分为非空子图集合 S 。令 $O_s = \{u : \phi(u) = s\}$ ，则操作覆盖与子图分配满足

$$\bigsqcup_{s \in S} O_s = O_c, \quad \sum_{k=0}^{N-1} \mathbf{1}[s \in \pi_k] = 1, \quad s \in S. \quad (4.1)$$

各序列内部也不允许重复出现子图。子图编号必须为非负整数，程序中的布尔值不作整数编号使用。COPY 操作由附件根据边界和空间需求处理，不与非 COPY 操作混入划分对象。

将跨子图数据依赖记为 E_{dep} ，将每个 π_k 中相邻子图的先后关系记为 E_{seq} 。合法序列要求

$$(S, E_{\text{dep}} \cup E_{\text{seq}}) \text{ 为有向无环图.} \quad (4.2)$$

仅分别检查数据依赖和各核序列还不够，因为两者合并可能构成相互等待。这里检查的是任务级联合图；把一个核心上的全部节点粗缩成一个点后出现的核间环，不直接等价于实际执行死锁。

4.2 依赖、管道、容量与带宽约束

对展开后的执行事件 e ，记开始和完成时刻为 σ_e, f_e ，依赖边上的规定释放延迟为 $\delta_{ee'}$ 。执行顺序满足

$$\sigma_{e'} \geq f_e + \delta_{ee'}, \quad (e, e') \in E_{\text{exec}}, \quad T^X(P) = \max_e f_e. \quad (4.3)$$

E_{exec} 包含原始数据依赖、合法排序与空间复用产生的依赖。每核四条 Pipe 各自串行，不同 Pipe 在依赖和空间允许时可并行。计算操作完成时间由其周期确定；传输持续时间则随同池请求数量变化。

设 $\mathcal{L}_{kr}(\theta)$ 为核心 k 在时刻 θ 实际占用存储 r 的张量集合， $v_e(\theta)$ 为传输请求的服务速率，则

$$\begin{aligned} \sum_{t \in \mathcal{L}_{kr}(\theta)} s_t &\leq C_r, \quad r \in \{L1, UB\}, \\ \sum_{e \in \mathcal{A}_p(\theta)} v_e(\theta) &\leq \beta_p, \quad p \in \{\text{DDR}, L2\}. \end{aligned} \quad (4.4)$$

L2 一项只在问题三开启。带宽池非空时，附件按活跃请求动态均分总带宽。L1/UB 空间不足时允许按固定机制换出和重读，实际驻留始终受容量约束。因此，静态生命周期峰值超容量说明可能需要换出，不能直接据此宣告方案不可执行。第七、八章的风险修正只影响候选优先级，式 (4.4) 仍是执行层面的硬约束。

表 4-1 固定硬件配置与场景差异

项目	取值或规则
核内存储	$C_{L1} = 524288$ byte, $C_{UB} = 131072$ byte
共享 DDR	$\beta = 60$ byte/cycle, 跨核共享总带宽
场景 A	每子图一个 Task; 同核切换 100 cycle; 跨核前驱 Task 完成后等待 1000 cycle
场景 B	每核一个合并 Task; 跨核 COPY_OUT 完成后等待 500 cycle, 再释放相应读取
只读 L2	1048576 byte, 250 byte/cycle, 逻辑张量索引、FIFO 淘汰
输出字段	node_to_subgraph, core_schedules

4.3 主目标、辅助目标与求解评价量

令 $\mathbf{X}$ 表示场景 A、场景 B 或场景 B 开启 L2 的配置。在对应可行域 $\mathcal{F}_{\mathbf{X}}(G, N)$ 内，优化方向为

$$\min_{P \in \mathcal{F}_{\mathbf{X}}(G, N)} (T^{\mathbf{X}}(P), D^{\mathbf{X}}(P)), \quad \text{以完工时间为主要目标.} \quad (4.5)$$

额外搬运 $D^{\mathbf{X}}$ 按附件统计相对于原图已有 COPY 的新增字节，包括边界处理与缓存换入换出。问题三中的 Cache 命中改变传输服务路径，逻辑 COPY 字节不一定随之减少。因此，额外搬运、DDR 实际服务量和缓存命中率分别统计。

直接求精确最优需要在大量划分和排列上展开执行。本文以图结构产生有限方案，并以 $J_A = \hat{T}_A$ 、 J_B 、 J_C 三种机制逐渐细化的评价量近似主要执行成本。其中通信与驻留修正都先由 byte 换算为 cycle，再参与排序。这些修正体现通信压力对时间的影响，所用系数属于算法参数，不改变题目给定的带宽、容量与等待时长。

最终输出由冻结的模型独立选择。官方执行程序在输出保存后计算 $T^{\mathbf{X}}$ 、 $D^{\mathbf{X}}$ 和缓存统计，用于报告真实结果。论文中的主要比较基于这些结果，而求解过程中的候选排序由自身模型完成。

第五章 数据预处理与结构特征分析

5.1 原始字段到计算依赖的转换

数据解析、程序实现及文稿整理使用 OpenAI Codex 辅助，队伍确认使用的模型为 GPT 5.6 SOL 和 GPT 6，另使用 Claude[9][10]。具体版本日期、Claude 的参与范围及人工复核记录见附录工具使用说明。

预处理按节点类型解析张量、计算操作和搬运操作，保留原始标识与字段。原始图在既有独立验收中通过附件的字段和图结构校验；本轮逐一核对 100 图 SHA256 与结构表记录一致。描述性统计读取原始字段，张量大小与操作周期均不重估，以便后续结果可追溯到同一输入。正式求解入口尚未覆盖的非法输入检查在第九章单列，不将离线数据核查等同于程序具备全部输入防护。

从操作集合中取出非 COPY 操作 O_c ，经张量连接的生产与消费关系投影为

$$G_c = (O_c, E_c), \quad E_c = \{(u, v) \in O_c^2 : \exists t \in T, (u, t), (t, v) \in E\}. \quad (5.1)$$

投影图用于判断操作先后及计算区域。张量标识、大小、生产者和消费者继续保存在索引表中，因此同一张量被多个操作消费时，可以按其实际共享关系计算搬运和驻留，而不把每条投影边都当作一份独立数据。原图中的 COPY 还用于识别输入读入与最终输出；投影不删除最终执行所需的搬运。

5.2 面向调度的结构量

对 G_c 进行拓扑排序，并在其无向形式上提取弱连通分量 $\mathcal{C} = \{C_1, \dots, C_K\}$ 。沿拓扑序计算深度与层宽，再累计各分量的双管道工作量：

$$\begin{aligned} d_u &= 1 + \max_{v \in \text{pred}(u)} d_v, & h &= \max_u d_u, \\ m_\ell &= |\{u : d_u = \ell\}|, & W_j^q &= \sum_{u \in C_j, p(u)=q} c_u, \quad q \in \{M, V\}. \end{aligned} \quad (5.2)$$

无前驱操作的深度为 1。入度大于 1 标识汇合位置，出度大于 1 标识分支位置。总工作量 $W = \sum_u c_u$ 与计算关键路径长度 L 由

$$\lambda_u = c_u + \max_{v \in \text{pred}(u)} \lambda_v, \quad L = \max_u \lambda_u, \quad \rho = W/L \quad (5.3)$$

得到，空前驱的最大值取零。 ρ 描述忽略通信和资源竞争后的工作量与路径长度之比，用于观察潜在并行结构；它不等于本题实际可取得的加速比。

还统计最大分量操作数占比 $q_{\max} = \max_j |C_j|/|O_c|$ 、向量工作量占比 $q_V = \sum_{p(u)=V} c_u/W$ ，以及具有两个及以上非 COPY 消费者的外部输入张量数。 $q_{\max}$ 按操作个数计算，与最大分量的周期占比不同。对于操作周期差异明显的图，后续装箱直接使用 W_j^M, W_j^V ，不以节点个数代替负载。

表 5-1 100 个原始计算图的描述性统计

统计量	最小值	中位数	最大值
非 COPY 操作数	552	3720.5	35705
操作依赖边数	400	4017	38493
弱连通分量数 K	1	39.5	2666
拓扑深度 h	3	47.5	2440
计算工作量 $W/cycle$	23052	509952	61275840
计算关键路径 $L/cycle$	400	6140	863040
工作量与路径比 ρ	3.42	43.13	6258.78

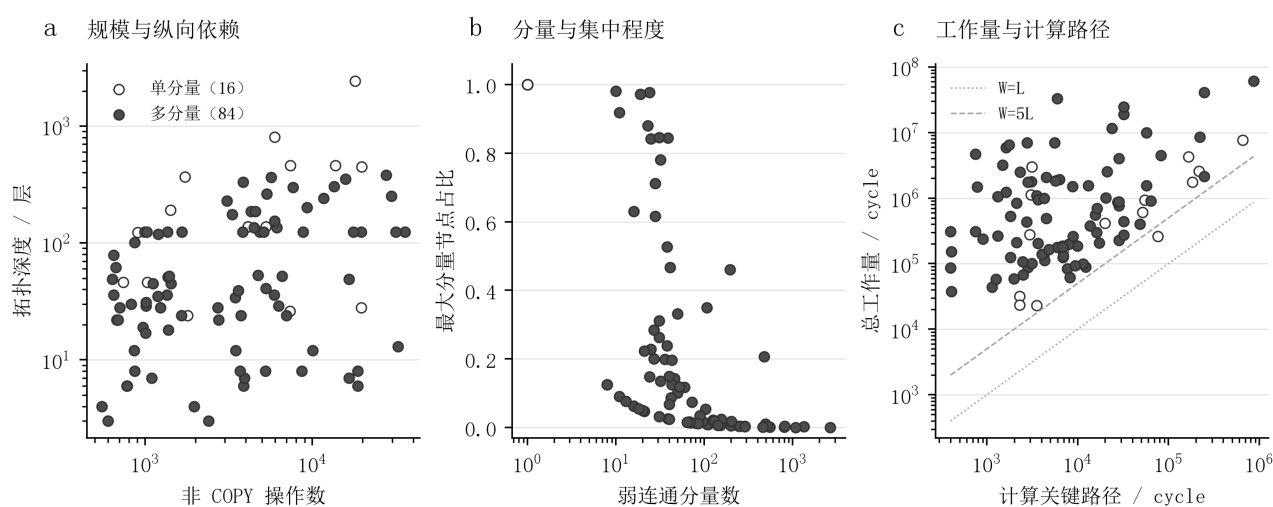


图 5-1 100 图的结构与工作量差异，每个点代表一张原始图。a 操作规模与拓扑深度；b 分量数与最大分量节点占比；c 计算关键路径与总工作量，参考线表示 $W=L$ 和 $W=5L$ 。空心、实心符号区分单分量和多分量图。对数坐标用于展示数量级跨度，不改变特征计算。

5.3 特征如何进入三问决策

84 个图具有多个弱连通分量，且其分量数均不少于 5；另有 16 个图只有一个分量。前一类通常提供直接分配独立计算块的机会，后一类若只保持分量完整，将无法利用多个核心。最大分量节点占比的中位数约为 0.122，也存在接近 1 的图，说明“分量很多”和“负载容易均衡”仍需分别判断。

共有 97 个图包含被多个计算操作消费的外部输入。这些输入是问题三共享读取的潜在来源，但它们也可能在同一核心内部完成复用，或因请求同时到达而无法命中 L2。此处统计仅用于识别可利用的数据关系；实际收益由第八章的事件模型及配对实验判断。

表 5-2 预处理特征与后续决策的对应关系

原图信息	计算量	对应建模作用
生产 - 消费依赖	分量、拓扑序、分支与汇合	第六章保留完整分量或选择内部切分位置
操作周期与管道	W_j^M, W_j^V	三问共用的双管道负载分配
深度和层宽	h, m_ℓ 、结构层间距	确定深度带尺度与层宽收缩边界
张量大小与消费者	边界字节、跨核消费集合	第六、七章的搬运估计与同核复用
存储位置与使用序列	活跃区间、峰值、下一次使用	第七、八章的 L1/UB 风险与换出估计
共享输入与访问事件	读取先后、FIFO 驻留窗口	第八章的读路径选择与缓存收益解释

结构特征在输入到达时重新提取。算法不把 100 图的均值、分位数或用例编号作为类别阈值；表 5-1 和图 5-1 用来解释输入差异，实际构造尺度由各图自身的深度、工作量及分支位置计算。这样的分工使数据分析与算法机理相互连接，同时保留了对新图进行结构分析的接口。

第六章 问题一：基于拓扑结构的切图与任务调度

6.1 由独立分量到内部切分的决策

场景 A 将子图边界转化为独立 Task 边界。第五章识别出的弱连通分量可以作为初始分配单位，在不切断内部依赖的条件下分散计算；单分量或负载高度集中的输入，则需要从内部切分获得并行机会。两类方案的取舍取决于新增并行工作量能否覆盖边界搬运和固定等待。

本问沿用式 (5.1) 的操作依赖图和式 (5.2) 的双管道工作量。先以完整分量装箱得到低边界代价方案，再从分支、汇合及层宽变化构造少量替代划分；时间估计负责比较这些结构选择，局部任务重排与合并用于处理优先候选中的等待和冗余边界。

6.2 场景 A 的约束与时间估计

对给定核心数 $N \in \{2, 3, 4, 5\}$ ，沿用子图映射 $\phi: O_c \rightarrow \mathcal{S}$ ，核心分配为 $\alpha: \mathcal{S} \rightarrow \{0, \dots, N-1\}$ ，核心 k 的子图执行序列为 π_k 。子图编号为非负整数；每个非 COPY 操作必须且只能分配到一个子图，每个子图必须且只能在一个核心序列中出现一次。子图依赖图以及加入同核先后关系后的联合图均须无环。输出采用附件规定的 `node_to_subgraph` 和 `core_schedules` 两个字段。

以系统完工时间为主要优化目标，同时考虑新增 DDR 搬运量。根据固定配置，同核相邻 Task 的等待时间为 $\delta_{\text{same}} = 100$ cycle，跨核前驱 Task 的等待时间为 $\delta_{\text{cross}} = 1000$ cycle；各核心共同使用带宽 $\beta = 60$ byte/cycle 的 DDR。L1 和 UB 的容量分别为 524288 byte 和 131072 byte。增加核心数不会增加 DDR 总带宽，同核 Task 切换也不能保留前一 Task 的缓存数据。

为在有限时间内比较候选，采用计算工作量、边界字节数和任务依赖共同构成时间估计。设 I_s 、 U_s 为 Task s 需要从 DDR 读入和写回 DDR 的张量集合，同一 Task 内按张量去重，则

$$\hat{B}_A = \sum_{s \in \mathcal{S}} \left(\sum_{t \in I_s} s_t + \sum_{t \in U_s} s_t \right), \quad \hat{d}_s = \max \left\{ W_s^M, W_s^V, \frac{\sum_{t \in I_s} s_t + \sum_{t \in U_s} s_t}{\beta} \right\}. \quad (6.1)$$

这里 W_s^q 按式 (5.2) 对 Task 内操作重新求和。两条计算管道和搬运可以重叠执行，故采用三类工作量所需时间的最大值估计 Task 时长。 $\hat{B}_A$ 记录任务边界的读写量；缓存不足产生的换入换出尚未进入该估计，后续官方验收将保留这一部分实际开销。

在子图依赖图中加入同核相邻 Task 的顺序边，记得到的前驱集合为 $P^\pi(s)$ 。沿该图的拓扑序递推：

$$\begin{aligned}\hat{f}_s &= \hat{d}_s + \max_{u \in P^\pi(s)} (\hat{f}_u + \delta_{us}), \\ \hat{T}_A &= \max \left\{ \max_s \hat{f}_s, \frac{\hat{B}_A}{\beta} \right\}, \quad \delta_{us} = \begin{cases} \delta_{\text{cross}}, & a(u) \neq a(s), \\ \delta_{\text{same}}, & u \text{ 为 } s \text{ 的同核紧邻前项}, \\ 0, & \text{其余同核依赖}. \end{cases}\end{aligned}\quad (6.2)$$

空前驱集合的最大值取 0。任务的预计启动时刻取决于前驱完成和同核队列的末端，两类约束在同一递推中合并。全局项 $\hat{B}_A/\beta$ 则避免将共享 DDR 当作各核独享资源。该模型省略了逐个 COPY 请求的动态带宽竞争，适合对有限候选作快速排序；结果部分使用方案冻结后获得的官方完工时间与搬运量。

6.3 分量装箱与结构切分

首先将弱连通分量作为完整物品分配。按 $\max(W_j^M, W_j^V)$ 从大到小处理分量，设核心 k 已分配的管道负载为 (L_k^M, L_k^V) ，则将当前分量放入

$$k^* = \arg \min_k \left(\max_{q \in \{M, V\}} (L_k^q + W_j^q), L_k^M + L_k^V, k \right). \quad (6.3)$$

括号内按字典序比较，选定后更新两条管道负载。同核分量合并为一个子图，空核保留空序列。该规则采用最长工作量优先分配思想[4]，并用双管道负载替代单一处理时间。共享 DDR 和缓存使本题不再满足经典独立作业调度模型，因而不直接援引其近似比作为本算法的性能保证。

完整分配保留了每个分量内部的生产 - 消费关系，分量之间也没有计算依赖，因而能够减少中间张量的跨核搬运。共享输入仍可能被多个核心分别读取，核内空间不足仍会触发换入换出。若一个分量集中了大部分计算量，完整分配还会使负载集中在单核上。内部切分候选主要用于处理这一类并行度受限的输入。

内部切分围绕三类结构展开。第一类保留首次汇合前的独立分支，或按照末端消费者对分支归组；第二类利用拓扑层宽的收缩与扩张确定阶段边界；第三类按照深度带与源端、末端可达关系划分。在深度带构造中，候选边界附近按跨层张量字节数选择切分位置，以减少将大张量切到不同 Task 的情况。上述构造均保持原操作和依赖，不拆分操作，也不复制计算。

基础深度带宽由当前图的深度 h 、总计算量 $W = \sum_{u \in O_e} c_u$ 和核心数确定：

$$w_0 = \max \left\{ 2, \left\lceil \frac{N\delta_{\text{cross}}h}{\max(1, W)} \right\rceil, \left\lceil \frac{Nh}{256} \right\rceil \right\}. \quad (6.4)$$

算法考察 $w_0, 2w_0, 4w_0$ 三种粒度。前一项避免过细切分，第二项使计算相对等待开销较小时倾向于更粗的 Task；第三项是固定的搜索规模控制规则，256 不是题目的硬件常数。此外，还保留减少活跃核心数的分量分配候选，以应对并行计算收益不足以抵消 DDR 争用的情形。

6.4 任务列表调度与有限合并

各结构候选先经过覆盖性、无环性检查和方案去重，再由式 (6.2) 排序。仅对排名靠前的至多三个候选进行任务调度与合并调整，以控制额外搜索量。

任务重排采用“就绪任务优先级 - 最早完成核心”的列表调度思想[5]。先自后向前计算

$$r_s = \hat{d}_s + \max_{v \in \text{succ}(s)} (\delta_{\text{same}} + r_v). \quad (6.5)$$

每次从全部已分配前驱的就绪 Task 中选取 r_s 最大者，比较其放到各核心后的预计完成时刻。设 F_k 是核心 k 已排任务的末端时间，则

$$\text{EFT}(s, k) = \hat{d}_s + \max \left\{ F_k + \delta_{\text{same}} \mathbf{1}[\pi_k \neq \emptyset], \max_{u \in \text{pred}(s)} (\hat{f}_u + \delta_{\text{cross}} \mathbf{1}[a(u) \neq k]) \right\}. \quad (6.6)$$

优先选择 EFT 最小的核心，并列时依次选择已排 Task 数较少、编号较小的核心。式 (6.5) 中的固定等待项只用于任务优先级；核心选择时则按式 (6.6) 区分同核与跨核等待。这里采用尾部追加排布，没有执行 HEFT 的空闲区间插入步骤。

合并调整只考虑同核相邻、且在 Task 拓扑序中也相邻的任务对。每轮按共享张量字节数选取至多八对，分别计算收缩后的方案；联合依赖图有环的方案被剔除，在其余方案中选取能够降低 $\hat{T}_A$ 的一项，最多执行两轮。任务重排与合并均从原结构候选独立出发，二者不串接。最终按预计时间、边界字节数和固定方案哈希依次比较；当完整分量方案与最优估计相差不超过 1% 时，保留完整分量方案。该阈值降低了选择结果对微小估计误差的敏感性。

算法 6-1 基于输入拓扑的切图与任务调度

输入：当前计算图 G ，核心数 N ，带宽 β ，等待参数 $\delta_{\text{same}}, \delta_{\text{cross}}$ 。

输出：子图映射 ϕ 与每核子图序列 $\pi_0, \dots, \pi_{N-1}$ 。

1. 读取张量、操作与边；提取非 COPY 操作集 O_c ，保留原始数据。
2. 按生产 - 消费关系构建 G_c ；计算拓扑序与节点深度 d_u 。
3. 提取弱连通分量、层宽、分支及汇合位置，统计双管道工作量与张量字节数。
4. 按式 (6.3) 构造完整分量方案 P_0 ，初始化待检验集合 $B \leftarrow \{P_0\}$ 。
5. 将首次汇合、末端分支与较少活跃核心数的分量装箱候选加入 B 。
6. 按式 (6.4) 计算 w_0 。
7. for $w \in \{w_0, 2w_0, 4w_0\}$ do
8. 构造数据感知深度带、分支汇合、层宽谷和末端可达归组候选，加入 B 。
9. end for；将宽度为 w_0 的源端可达归组候选加入 B 。
10. 逐项检验 B ，将覆盖完整、联合图无环且不重复的方案组成 P ；记录拒绝原因。
11. 按式 (6.1) - (6.2) 评价 P ；按时间、字节数截取至多三个原始候选。
12. for 所选方案中的每个 P do
13. 从 P 分别生成就绪列表重排方案与至多两轮的相邻任务合并方案。
14. 重新检查合法性，计算时间估计，将不同的合法方案加入 P 。
15. end for
16. 取 $\hat{T}_A$ 最小的 P^* ；时间并列时比较边界字节数与固定方案哈希。
17. 若 $\hat{T}_A(P_0) \leq 1.01\hat{T}_A(P^*)$ ，令 $P^* \leftarrow P_0$ 。
18. return P^* 的 node_to_subgraph 与 core_schedules。

6.5 求解结果与分析

算法对每个图、每种核数分别从原始输入生成方案。所有全局参数在正式运行前固定，生成过程中不读取历史方案和逐图得分，也不调用官方评估器。方案保存后，再使用未修改的附件程序完成 100 个图、2 - 5 核共 400 组验收，全部通过。

设第 i 个图在 N 核下的官方完工时间为 $T_{i,N}$ ，按题目规定采用的单核整图时间为 $T_{i,1}$ 。统计采用逐图加速比的算术平均：

$$S_{i,N} = \frac{T_{i,1}}{T_{i,N}}, \quad \bar{S}_N = \frac{1}{100} \sum_{i=1}^{100} S_{i,N}. \quad (6.7)$$

单核加速比按定义置为 1，不另行进行单核搜索。表 6-1 给出全部用例的汇总；额外搬运采用官方 `added_copy_bytes` 字段，并按 $1\text{MiB} = 2^{20}\text{ byte}$ 换算。

表 6-1 问题一的全量求解结果

核心数	平均加速比	加速比中位数	平均额外搬运 / MiB
2	1.8648	1.9583	13.848
3	2.6016	2.7753	14.514
4	3.2650	3.5074	15.557
5	3.8311	4.0428	15.285

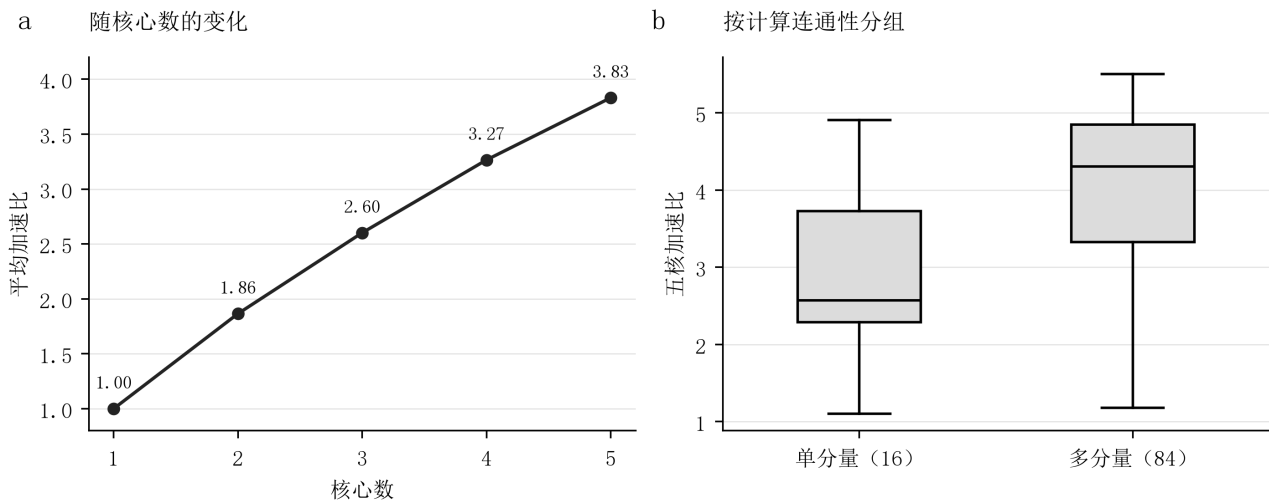


图 6-1 问题一的加速比结果。左图为每种核数下 100 图的算术平均，单核值按定义为 1；右图为五核下按弱连通分量数分组的分布。箱体表示四分位区间，中线为中位数，须延至 1.5 倍四分位距内的最远观测，点为其外观测。

五核平均加速比为 3.8311。多分量图的五核平均值为 4.0041，单分量图为 2.9228，表明输入结构对可利用的并行性有明显影响。多分量允许在较少新增依赖的条件下分配负载；单分量则通常需要在并行化与边界代价之间作出取舍。这一差异与独立分量提供可直接分配计算块的机制一致，分组内仍存在较大离散，故还需观察逐图对照。

与仅采用完整分量装箱的同批五核方案相比，增加结构切分与有限调度调整后，38 个图耗时下降、2 个图上升、60 个图不变。改进集中在完整分量分配以外的候选被选中后取得收益的输入上，少数退步反映了代理排序与实际执行的差异。五核平均额外搬运中，缓存换入换出占 7.423 MiB；任务边界之外的核内空间开销已达到不可忽略的规模。场景 B 延长了同核张量的可驻留范围，对这部分开销需要进一步建模。

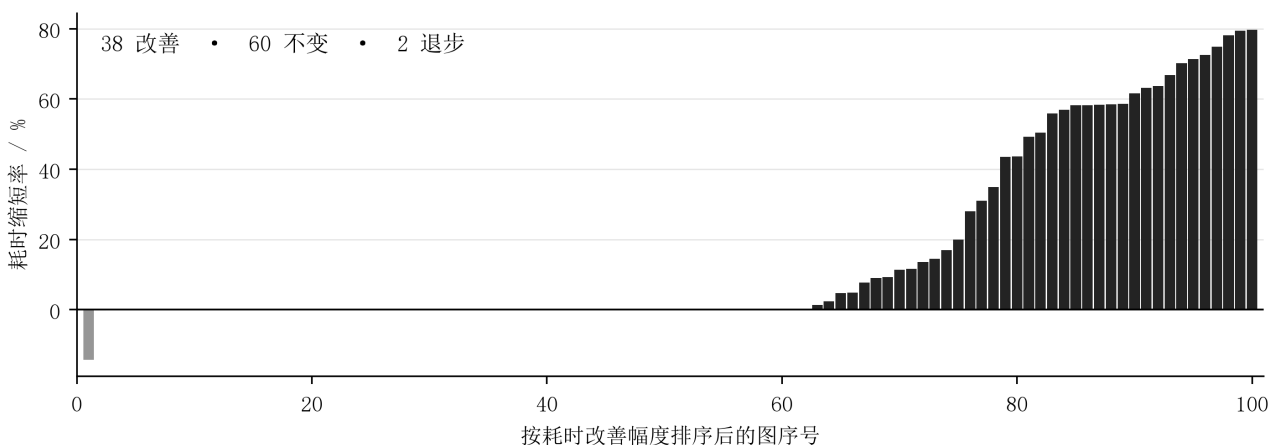


图 6-2 五核下相对于完整分量装箱的逐图耗时变化。横轴为全部 100 图按改善幅度排序后的序号，纵轴为 $100(1 - T_A/T_{\text{component}})$ ；零线以上表示扩展结构候选后的最终方案更快。该对照在同一场景 A 下验收，用于观察候选扩展及选择的综合作用。

图 6-2 保留全部未改善和退步的用例。大量零变化说明在这些图上，低边界代价的分量分配已经足以被最终规则保留；正向尾部反映了内部切分和任务顺序调整的价值。两个退步用例则提示，有限候选中的代理最优仍可能低于分量对照的实际表现，继续增加构造数量并不能自动修复这一排序误差。

第七章 问题二：通信与缓存驻留联合约束下的切图调度

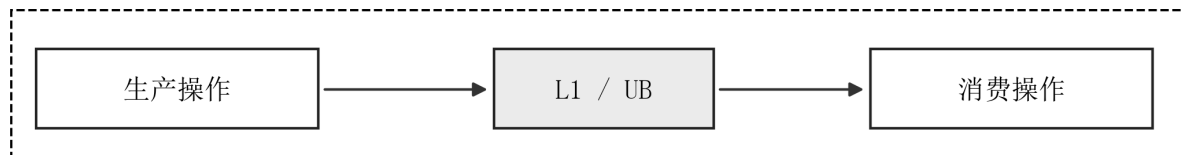
7.1 场景 B 的运行机制与建模重点

场景 B 将同一核心的全部子图合并为一个 Task，同核生产者与消费者之间可以保留 L1/UB 中的数据。此时，决定边界通信的因素由“是否跨子图”转变为“是否跨核心”。跨核依赖仍需 COPY_OUT 和 COPY_IN，目标搬入须等待源搬出完成，并经过 $\delta_B = 500$ cycle 的同步延迟。图 7-1 对比了两种场景的数据路径。

a 场景 A：同核跨 Task 仍经过 DDR



b 场景 B：同核子图合并，保留可驻留数据



同一核心的一个 Task；容量不足仍可能发生换出

图 7-1 两种场景下的 Task 边界与数据路径。图中为规则示意，未按实测时间绘制；L1、UB 的容量限制及共享 DDR 总带宽均保持题目配置。

合并 Task 后，中间张量可以在生产操作结束后保留到同核消费者使用。通信量因而有机会减少，张量占用核内空间的时间也可能延长。将共享大张量的操作安排在同一核心，需要同时考虑该核的计算负载与驻留数据规模。本问据此在分核阶段估计通信，在候选排序阶段补充缓存风

险，仍采用有限种结构构造控制计算成本。

沿用第六章的 $\mathbf{G}_c$ 、 ϕ 、 $\mathbf{a}$ 、 π_k 及硬件符号。每个 case 都重新计算拓扑序、分量、层宽和分支汇合位置，不读取问题一已输出的切图方案。记分支或汇合节点所在的不同拓扑层为 $\ell_1 < \dots < \ell_m$ ，其相邻间距集合为 $\mathcal{D} = \{\ell_{j+1} - \ell_j\}$ 。候选深度带宽取

$$w_B = \begin{cases} \max\{2, \min\{32, \text{round}(\text{median } \mathcal{D})\}\}, & \mathcal{D} \neq \emptyset, \\ \max\{2, \min\{32, \lceil \sqrt{h} \rceil\}\}, & \mathcal{D} = \emptyset. \end{cases} \quad (7.1)$$

分支与汇合层相邻间距刻画了当前图中重复出现的结构尺度，取其中位数可减少少数极长链段对带宽选择的影响。式中的 round 采用最近整数、半整数取偶的规则，与实现一致；2 和 32 为固定的搜索尺度边界。这里的深度带宽以拓扑层数计，硬件配置仍按题目给定值读取。

7.2 跨核通信与完成时间估计

令 P_t 、 C_t 分别表示张量 t 的生产核集合和消费核集合；令 $\mathbf{e}_t = \mathbf{1}$ 表示原图包含该张量的最终 COPY_OUT。为比较分核方案，按生产核 - 消费核对累计跨核写出与读入；同核消费继续沿用核内数据。相应的计数为

$$\begin{aligned} n_t^{\text{in}} &= \mathbf{1}[P_t = \emptyset] |C_t|, \\ n_t^{\text{out}} &= \mathbf{1}[P_t \neq \emptyset] \mathbf{1}[\mathbf{e}_t = \mathbf{1} \text{ 或 } C_t = \emptyset] |P_t|, \\ n_t^{\text{cross}} &= \sum_{k \in P_t} \sum_{l \in C_t} \mathbf{1}[k \neq l]. \end{aligned} \quad (7.2)$$

由此得到缓存换出之前的通信量估计

$$\hat{B}_B = \sum_{t \in T} s_t (n_t^{\text{in}} + n_t^{\text{out}} + 2n_t^{\text{cross}}). \quad (7.3)$$

同一核心上多个操作读取同一张量时，在集合 C_t 中只记录一个消费核，避免重复累计该核的边界读取。 $\hat{B}_B$ 包含输入读入、结果写出和跨核传递的估计工作量。结果表中的额外搬运则按附件扣除原图已有 COPY、加入实际缓存换入换出后统计，两者的计数范围不同。

记核心 k 的管道负载为 $L_k^q = \sum_{u: a(\phi(u))=k, p(u)=q} c_u$ ， b_{uv} 为操作依赖 $u \rightarrow v$ 关联的张量字节数之和。沿操作拓扑序递推：

$$\begin{aligned} \hat{f}_v^B &= c_v + \max_{u \in \text{pred}(v)} \left[\hat{f}_u^B + \mathbf{1}[a(\phi(u)) \neq a(\phi(v))] \left(\delta_B + \frac{2b_{uv}}{\beta} \right) \right], \\ \hat{T}_B &= \max \left\{ \max_{k,q} L_k^q, \max_v \hat{f}_v^B, \frac{\hat{B}_B}{\beta} \right\}. \end{aligned} \quad (7.4)$$

最忙管道、最长依赖路径和共享 DDR 总工作量分别给出三种时间尺度，取其最大值作为候选的主要耗时估计。各项统一为 cycle，便于与后续缓存风险的时间修正相加。四条 Pipe 的动态配合与请求间带宽均分没有在该式中逐事件展开，实际耗时仍需由冻结方案的官方执行结果检验。

7.3 张量驻留区间与缓存风险

为识别同核合并可能带来的缓存压力，先按 π_k 连接各子图，并在子图内保留原操作拓扑序，得到核心 k 的分析序列。对张量 t ，令 f_{tk} 、 l_{tk} 分别为其生产者或消费者在该序列中第一次和最后一次出现的位置。对资源 $r \in \{L1, UB\}$ ，静态活跃字节峰值为

$$H_k^r = \max_j \sum_{t: \text{pos}_k(t)=r} s_t \mathbf{1}[f_{tk} \leq j \leq l_{tk}], \quad R_B = \sum_{r \in \{L1, UB\}} \left(\max_k H_k^r - C_r \right)_+. \quad (7.5)$$

其中 $(x)_+ = \max(x, 0)$ ， C_r 为对应缓存容量。在静态估计中，原 DDR 输入按 UB 驻留处理。程序通过区间起止位置的差分累计计算 H_k^r ，无需按实际周期展开整个执行过程。

静态区间记录的是分析序列中的位置。附件执行时，输出空间在操作发射前申请，输入空间在最后一次使用完成后释放，并按固定核内机制插入换入换出。两种时间表示的差异会影响同时驻留的张量集合。 R_B 因此作为长期驻留的风险量进入排序；即使其值为零，实际执行中仍可能产生缓存换出。容量可行性在原版执行程序中按 L1/UB 的硬上限检查。

对合法候选采用统一评分

$$J_B = \hat{T}_B + \frac{\hat{B}_B + \gamma_B R_B}{\beta}, \quad \gamma_B = 10. \quad (7.6)$$

在最大瓶颈时间相近时，附加 DDR 工作量使通信较少的方案优先，缓存风险项则降低长期占用核内空间的方案的优先级。这两个修正项与 $\hat{T}_B$ 均以 cycle 表示，构成用于排序的评价函数。开发阶段的 28 个正风险候选中，实际换出相关字节与风险量之比的中位数为 10.54，据此将全局系数 γ_B 取为 10 并冻结。该系数适用于本轮统一排序，各输入图只重新计算其 R_B 。

7.4 有限候选构造与算法实现

问题二构造五类方案：完整分量装箱、层宽谷切分、宽度为 w_B 和 $2w_B$ 的两种数据感知深度带，以及宽度为 w_B 的固定深度带。各方案使用同一输入依赖和张量字节数，其中分量或带内独立区域采用第六章的双管道负载分配规则。数据感知切分优先寻找跨层字节数较少的位置，层宽谷切分则利用并行度收缩处形成阶段。五类方案生成后去重，重复结构不再评分。

候选生成后，先检查节点覆盖、子图唯一分配及联合依赖图无环性。通过检查的不同方案才计算通信、路径与缓存项，并按 J_B 排序；同分时依次比较 $\hat{B}_B$ 和固定的候选类别顺序。算法 7-1 给出这一过程，输出仍为各非 COPY 操作的子图编号与各核子图序列。

算法 7-1 基于输入结构与驻留风险的场景 B 调度

输入：当前计算图 G ，核心数 N ，固定配置 $\beta, \delta_B, C_{L1}, C_{UB}$ 。

输出：子图映射 ϕ 与每核子图序列 $\pi_0, \dots, \pi_{N-1}$ 。

1. 读取当前 case，构建 G_c 、张量生产者/消费者表及大小、存储位置索引。
2. 计算拓扑序、深度、弱连通分量、层宽和分支汇合层；按式 (7.1) 得到 w_B 。
3. 生成分量装箱、层宽谷、两种数据感知深度带及固定深度带共五类候选。
4. 初始化记录集合 $\mathcal{R} = \emptyset$ 与已处理方案哈希集合。
5. for 按固定类别顺序生成的每个候选 P do
6. 校验节点覆盖、子图覆盖及依赖与核内顺序的无环性；失败则记录原因并跳过。
7. 计算完整方案哈希；若重复则跳过，否则加入已处理集合。
8. 由 ϕ, a 建立 P_t, C_t ，计算 $\hat{B}_B, L_k^q$ 和 $\hat{T}_B$ 。
9. 按每核子图序列构造分析序列，计算 f_{tk}, l_{tk} ，以差分累计得到 H_k^r 。
10. 计算 R_B 和 J_B ，将候选及各项评价量加入 $\mathcal{R}$ 。
11. end for
12. 若 $\mathcal{R}$ 为空则报告失败；否则按 $(J_B, \hat{B}_B, \text{类别顺序})$ 取最小项。
13. return 对应的 node_to_subgraph 与 core_schedules。

7.5 求解结果与分析

按照与问题一相同的单核参照和统计口径，对 100 个计算图的 2-5 核方案进行独立验收，共 400 组，全部成功。表 7-1 列出两种场景在相同图和核数下的结果。第九章另列独立输入检验，以区分给定图表现与未见结构上的稳定性。

表 7-1 问题一与问题二的全量结果比较，额外搬运单位为 MiB

核心数	问题一加速比	问题二加速比	问题一额外搬运	问题二额外搬运
2	1.8648	1.9358	13.848	10.076
3	2.6016	2.7452	14.514	8.683
4	3.2650	3.4645	15.557	8.569
5	3.8311	3.9522	15.285	9.756

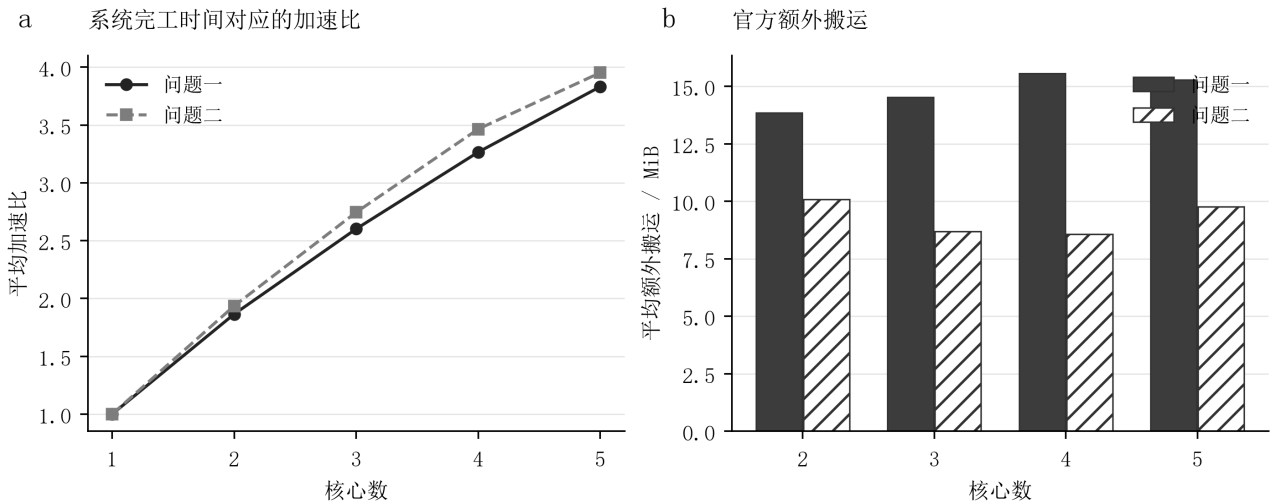


图 7-2 两种场景的全量结果比较。左图为逐图加速比的算术平均，单核值按定义为 1；右图为官方统计的平均新增 DDR 字节数，以 MiB 表示。除单核定义值外，每个点或柱均对应 100 个计算图。

问题二在 2-5 核下的平均加速比均高于问题一，五核均值由 3.8311 增至 3.9522。同时，五核平均额外搬运由 15.285 MiB 降至 9.756 MiB。场景 B 允许保留同核数据，为降低边界搬运提供了条件；但两场景使用了各自适配的方案，以上差异同时包含场景规则和方案选择的影响，不能全部归因于单一算法步骤。

逐图比较中，五核时有 31 个图耗时下降、19 个图上升、50 个图相同。以问题一的逐图耗时为分母，平均耗时缩减率为 1.67%。问题二的五核平均缓存换入换出新增字节仍有 5.315 MiB，表明同核复用的收益受到驻留容量制约。静态风险量未记录每次实际申请和释放的时刻，候选排序仍可能低估局部高峰及其搬运代价。

本问以五类构造限制了每次求解的候选规模，输入适应性来自当前图的拓扑层间距、管道负载和张量区间。后续引入共享只读 Cache 后，同一读请求可能占用不同的带宽池，命中还取决于读取完成与淘汰的先后顺序。第八章将这部分时序展开，用于进一步区分静态通信量相近的方案。

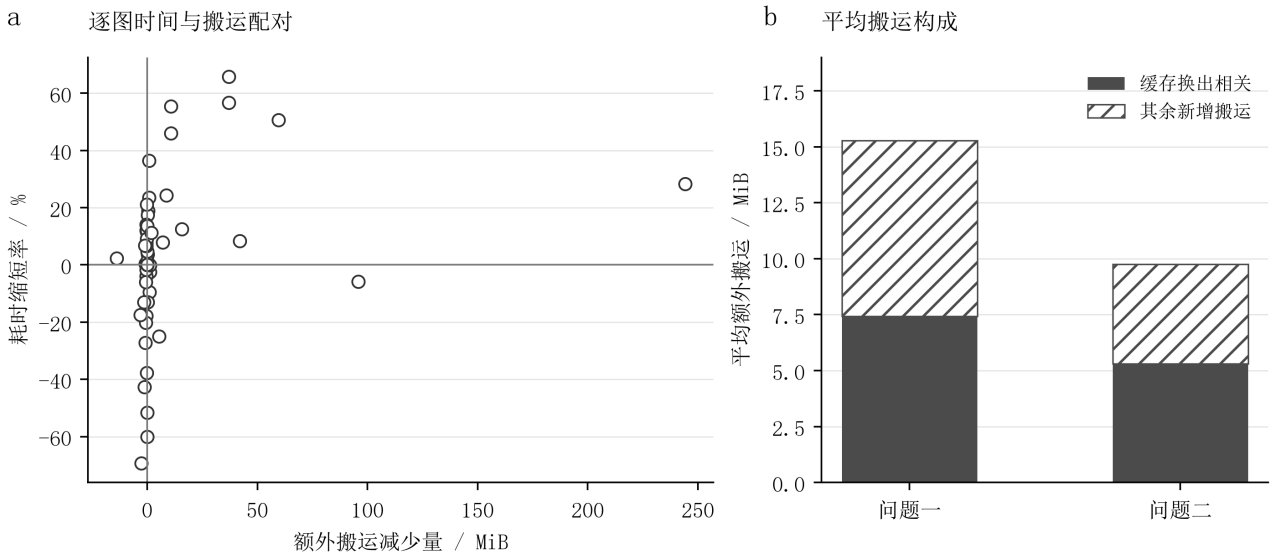


图 7-3 五核下问题二相对问题一的逐图变化，每个点对应相同输入。横轴为额外搬运减少量，纵轴为耗时缩短率；右上表示两项同时改善，右下表示搬运减少但耗时增加。右侧分解两场景的平均额外搬运为缓存换出相关部分与其余部分，分解采用官方记录。两场景均使用各自求得的方案。

五核额外搬运均值减少约 36.17%，但逐图平均耗时只缩短约 1.67%，两者不能按同比例换算。图 7-3 直接展示了搬运量与时间变化的配对关系：实际收益还取决于搬运是否占据完成路径、是否与计算重叠，以及分核后各管道的负载。其中有 7 个图在额外搬运减少时仍出现耗时增加。对于这些图，应优先检查这些时间关系，而不能仅以总字节数下降判定调度改善。

问题二的静态评分适合快速过滤通信与驻留压力大的方案，却没有逐请求展开两类活动的重叠。第三问保留这一数据关系，再对读取到达、命中与带宽共享建立事件描述，以减少静态工作量相近时的选择盲区。

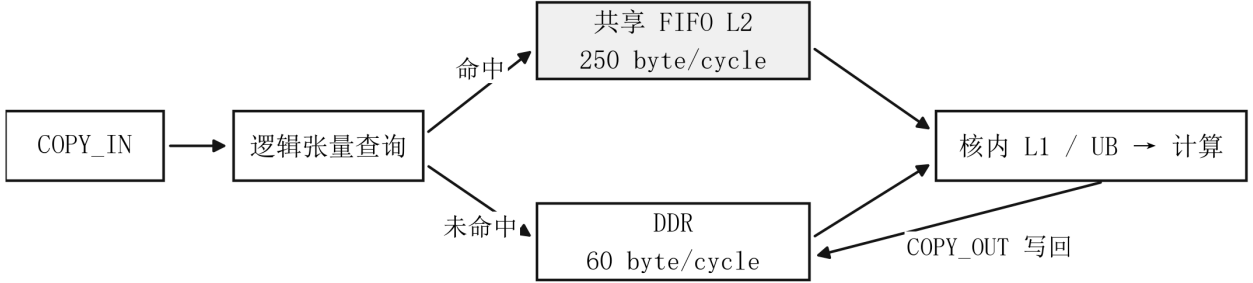
第八章 问题三：共享只读 Cache 下的切图与协同调度

8.1 缓存数据通路与协同调度条件

问题三在场景 B 的基础上增加所有核心共享的只读 FIFO Cache。同核子图仍合并为一个 Task，跨核传递仍经过 COPY_OUT、COPY_IN 和 $\delta_B = 500$ cycle 的同步延迟。新增 Cache 容量为 $C_{L2} = 1048576$ byte，读带宽为 $\beta_{L2} = 250$ byte/cycle，与带宽为 $\beta = 60$ byte/cycle 的 DDR 分别结算。L1、UB 继续承担核内计算数据的驻留，两者容量沿用第六章配置。

读请求的服务路径取决于发射时的 Cache 状态。COPY_IN 按逻辑张量标识查询，命中时占用 L2 读带宽，未命中时占用 DDR 带宽；COPY_OUT 始终进入 DDR。一次读取完成后，附件程序尝试将对应张量插入 FIFO 队列，空间不足时先淘汰最早插入的条目。命中读取不会调整已有条目的队列位置。图 8-1 给出数据通路及插入、淘汰之间的关系。

a 读写路径：请求发射时确定带宽池



各核心共享总带宽；同一池内在途请求均分，两个池独立计时。

b 读取完成：检查插入条件，必要时从队首淘汰

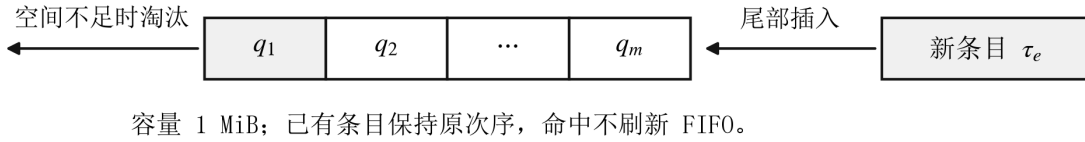


图 8-1 共享只读 Cache 的数据通路。DDR 与 L2 分别提供 60、250 byte/cycle 的总带宽，同一池内在途请求动态均分带宽。下部按 FIFO 次序表示读取完成后的插入处理，属于规则示意，框宽与箭头长度不代表实际时间或张量大小。

缓存能够减少部分 DDR 读取等待，其收益还取决于请求到达时间和张量能否保留到下一次访问。例如，多个核心同时首次读取同一张量时，早先的读取尚未完成，后续请求仍可能全部未命中；张量即使命中过，之后也会随着新条目插入而被淘汰。调度方案需要同时安排计算负载和读取先后，单独累计共享输入字节无法确定实际收益。

沿用 G_c 、 ϕ 、 a 与 π_k ，记完整方案为 $P = (\phi, a, \pi_0, \dots, \pi_{N-1})$ 。可提交的决策仍是切图映射与核心序列，Cache 命中、指令发射及淘汰时间随执行产生。本文通过分核和合法重排形成读取机会，不增加预热操作、等待指令或起始时间字段。整体目标保持完工时间优先，并在预计耗时相同时比较搬运工作量。

8.2 当前输入图上的有限候选构造

求解从当前图重新计算弱连通分量、拓扑深度、双管道工作量及张量生产 - 消费关系。完整分量装箱沿用式 (6.3)，在同一分核映射下保留两种子图表达：每核合并为一个子图，以及每个独立分量保留为一个子图。两种表达最终均按场景 B 合为核内 Task，传入的子图次序会参与核内展开，因此分别纳入候选。

对共享输入较多的分量，还构造带输入亲和性的装箱方案。设分量 C_j 的外部输入集合为 I_j ，核心 k 已分配分量的外部输入并集为 I_k^{core} ，分配优先量取为

$$F_{jk}^{\text{aff}} = \max_{q \in \{M, V\}} (L_k^q + W_j^q) + \frac{\sum_{t \in I_j \setminus I_k^{\text{core}}} s_t}{\beta}. \quad (8.1)$$

第一项表示分配后最忙计算管道的工作量，第二项折算新增外部输入的读取时间。选择 F_{jk}^{aff} 较小的核心，使已经分配的共享输入具有继续同核使用的机会。并列时依次比较该核原有总工作量与核心编号。这里采用 DDR 带宽估算输入压力，具体读取是否命中则交由事件模型判断。

内部结构切分继承前两问的分支、汇合与深度带构造。以式（7.1）得到的拓扑尺度 w_B 为起点，再计入本图计算量与同步开销的比例，令

$$w_C = \min \left\{ h, \max \left[2, w_B, \left\lceil \frac{N\delta_B h}{\max(1, W)} \right\rceil \right] \right\}, \quad W = \sum_{u \in O_c} c_u. \quad (8.2)$$

w_C 以拓扑层数计；分母中的 1 按一个 cycle 理解，用于零工作量输入的数值保护。此次附件的非 COPY 操作周期均为正，输入契约仍按非负值检查。当计算量相对同步代价较小时，该式增大切分尺度，避免大量短计算段被固定等待隔开。数据感知深度带考察 w_C 和 $2w_C$ 两种粒度，其余构造保持固定数量。

表 8-1 问题三的候选构造及其作用

构造	数量	使用当前图的信息与处理方式
完整分量装箱	2	按双管道工作量分核；分别采用同核合并、独立分量保留两种表达
输入亲和装箱	1	按式（8.1）兼顾管道负载与新增输入读取
汇合前分支、共享前缀	2	利用首次汇合或末端消费者关系组织独立计算区域
分叉 - 汇合阶段、层宽谷	2	按分支关系或层宽收缩位置确定阶段边界
数据感知深度带	2	在 w_C 、 $2w_C$ 尺度内优先选择跨层字节较少的边界
独立分量复用重排	1	保持分核结果，调整同核独立分量的执行先后

复用重排只作用于彼此独立的分量。核心 0 首先安排“输入字节数除以 DDR 带宽，再加双管道最大工作量”较小的分量；其余核心的首个分量优先减少与该输入集合的重合，并在同分时优先较大工作量。各核后续分量按与上一个分量共享的输入字节量降序安排，再以工作量和分量编号确定并列次序。这个顺序尝试减少启动阶段的重复未命中，并缩短后续共享读取之间的间隔。真正的完成先后仍受管道和依赖影响，核心 0 没有额外的预热权限。

表 8-1 至多产生十份候选，结构相同的方案在本次调用中去重。各候选共享相同硬件参数与评价规则，构造过程无需按用例编号选择算法，也不读取前两问的已输出方案。

8.3 双带宽池与 FIFO 事件模型

为比较不同读取顺序，先将候选展开为轻量事件图。每个非 COPY 操作保留其计算周期 c_u ；外部输入和跨核传递形成读事件，跨核生产端及最终输出形成写事件。跨核写完成到读释放之间保留 δ_B 延迟。各核按子图序列连接局部拓扑序，得到操作位置 j_u ；计算事件取排序键 $3j_u + 1$ ，服务消费者的读事件取 $3\min j_u$ ，服务生产者的写事件取 $3\max j_u + 2$ 。同一 Pipe 内按该键与事件编号加入先后边，四条 Pipe 各保留一个执行槽。

这些边与原数据依赖共同确定事件的释放条件。若两类顺序合并后出现环，该候选在代理模型中被拒绝，并记录具体原因。局部顺序由当前图直接展开，官方 Step 1 - 3 的完整空间复用补边未被复制进求解器，核内次序差异因而成为估计误差的一个来源。

设 $\mathcal{A}_p(\theta)$ 为时刻 θ 在带宽池 $p \in \{\text{DDR}, \text{L2}\}$ 中传输的请求集合， $r_e(\theta)$ 为请求 e 尚未服务的字节数。其发射时依据逻辑张量标识 τ_e 查询缓存，记查询前状态为 $\mathcal{Q}_e^{\text{issue}}$ ，则

$$z_e = \mathbf{1}[\tau_e \in \mathcal{Q}_e^{\text{issue}}], \quad p_e = \begin{cases} \text{L2}, & e \text{ 为读请求且 } z_e = 1, \\ \text{DDR}, & \text{其余搬运请求}. \end{cases} \quad (8.3)$$

同一时间戳内已经处理的完成和插入事件应计入 $\mathcal{Q}_e^{\text{issue}}$ 。请求发射后，其带宽池保持不变，队列中的条目之后是否被淘汰不会改变这次在途传输的服务路径。

在集合 $\mathcal{A}_p$ 不变的时间段内，池内请求均分该池带宽：

$$r_e(\theta + \Delta\theta) = r_e(\theta) - \frac{\beta_p}{|\mathcal{A}_p(\theta)|} \Delta\theta, \quad \beta_{\text{DDR}} = \beta, \quad \beta_{\text{L2}} = 250 \text{ byte/cycle}. \quad (8.4)$$

对非空带宽池，下一个传输完成事件距当前时刻的间隔为

$$\Delta_p = \frac{|\mathcal{A}_p(\theta)|}{\beta_p} \min_{e \in \mathcal{A}_p(\theta)} r_e(\theta). \quad (8.5)$$

模型在就绪释放、计算完成和两池传输完成三类事件中选择最早者，按式（8.4）同步扣减所有在途请求的剩余字节。请求加入或完成后重新计算 Δ_p 。由此，各池在非空时段内的总服务速率恰为自身带宽，DDR 与 L2 的服务量不会混入同一个容量约束。

FIFO 队列记为 $\mathcal{Q} = (q_1, \dots, q_m)$ ， q_1 最早插入。读取完成时，若 τ_e 尚未驻留且 $\setminus(0$

$$d^* = \min \left\{ d \in \{0, \dots, m\} : \sum_{r=d+1}^m s_{q_r} + s_{\tau_e} \leq C_{\text{L2}} \right\}, \quad \mathcal{Q}^+ = (q_{d^*+1}, \dots, q_m, \tau_e). \quad (8.6)$$

若该标识已经存在，则保持原队列；张量超容量或大小为零时不插入。式（8.6）每次从最早条目开始释放空间，插入后的总字节数不超过 C_{L2} 。附件在每次 COPY_IN 完成时都尝试这一处理。因此，一次发射时命中的读取，如果其条目在传输期间被淘汰，也可能在完成时再次插入。张量的有效驻留窗口由实际插入与淘汰事件确定，访问次数本身不会刷新 FIFO 次序。

算法 8-1 候选方案的双带宽池事件评价

输入：候选事件图 $\mathcal{H}(P)$ ， $\beta, \beta_{\text{L2}}, C_{\text{L2}}$ 及边上的释放延迟。

输出：预计完工时间 $\hat{T}_{\text{evt}}$ 、DDR 与 L2 服务字节数。

1. 置时刻 $\theta \leftarrow 0$ ；初始化剩余前驱数、就绪队列和完成事件队列。
2. 置 $\mathcal{Q} \leftarrow \emptyset$ ， $\mathcal{A}_{\text{DDR}} \leftarrow \emptyset$ ， $\mathcal{A}_{\text{L2}} \leftarrow \emptyset$ 。
3. while 尚有事件未完成 do
4. 取出释放时刻不晚于 θ 的就绪事件；读写按式（8.3）入池，计算按 c_u 登记完成时刻。
5. 由式（8.5）求两池最近完成间隔，空池取 $+\infty$ 。
6. 取就绪释放、计算完成和传输完成中的最早时刻 θ' ；若均不存在，报告代理依赖环。
7. 按 $\Delta\theta = \theta' - \theta$ 扣减两池剩余字节，置 $\theta \leftarrow \theta'$ ，收集本时刻已完成事件。
8. for 按固定事件编号排序的每个完成事件 e do
9. 若 e 为读事件，按式（8.6）处理完成插入或保持原队列。
10. 更新后继剩余前驱数与最早释放时刻；前驱全部完成的后继加入就绪队列。
11. end for
12. end while
13. return θ 与累计服务字节数。

算法 8-1 计算的是既定代理事件图上的完工时间。重复读命中、FIFO 淘汰和带宽动态均分在这个图上逐事件更新；实际核内顺序与换出插入仍由附件决定。事件次序决定带宽池的占用与 FIFO 驻留；核内空间展开的近似误差则通过最终验收及逐图差异分析观察。

8.4 核内驻留修正与方案选择

共享 L2 的读取加速不会扩大 L1/UB 的核内容量。对每个候选，另按各核分析序列扫描张量的产生、使用和释放。当前操作关联的输入、输出张量被标为受保护集合，其余驻留张量在空间不足时才可换出。设 $\nu_t(j)$ 为张量 t 在当前位置 j 之后的下一次使用位置，无后续使用时取 $+\infty$ ；从可逐出集合 $\mathcal{E}$ 中选择

$$t^* = \arg \max_{t \in \mathcal{E}}^{\text{lex}} (\nu_t(j), s_t, \text{id}(t)). \quad (8.7)$$

若未来仍需使用且模型中尚无持久副本，计入一次写回；已经访问而不再驻留的张量再次读入时，计入读取字节。最后一次使用后释放空间，逐核累计得到 $\hat{B}_{\text{spill}}$ 。原 DDR 张量在这个静态驻留分析中归入 UB，与第七章的估计保持一致。

上述扫描采用顺序区间和下一次使用位置，未展开空间分配造成的全部同步边。若受保护工作集使估计无法继续逐出，记录超容量量，并向已累计流量加入其两倍作为风险修正。所得字节数用于候选排序，不具有实际搬运量上界的含义。正式执行中的容量满足情况仍由原版调度和换入换出机制检验。

令 $\hat{B}_{\text{svc}}$ 为事件模型中的 DDR 与 L2 服务字节之和，候选的主要评价量为

$$J_C(P) = \hat{T}_{\text{evt}}(P) + \gamma_C \frac{\hat{B}_{\text{spill}}(P)}{\beta}, \quad \gamma_C = 1. \quad (8.8)$$

事件图已经包含边界传输及其与计算的重叠，第二项补入图外核内换出带来的时间压力。采用 DDR 带宽作换算，使周期与字节的修正保持同一量纲。开发阶段对一个全局系数比较 0, 0.25, 0.5, 1, 2, 4 六个取值后，将 γ_C 固定为 1；本次求解只更新当前图产生的两项估计。

在通过结构检查且能够完成代理评价的集合 $\mathcal{P}(G, N)$ 中，按

$$P^* = \arg \min_{P \in \mathcal{P}(G, N)}^{\text{lex}} (J_C(P), \hat{B}_{\text{svc}}(P) + \hat{B}_{\text{spill}}(P), \kappa(P)) \quad (8.9)$$

选择输出，其中 $\kappa(P)$ 为候选名称的固定字典序。完工时间修正量决定主要次序，搬运量只处理主要评分相同的情况，最后一项保证重复运行时选择一致。该规则在候选集合内作确定性比较，计算量受构造数限制。

算法 8-2 输入图自适应的只读 Cache 协同调度

输入：当前原始图 G ，核数 $N \in \{1, \dots, 5\}$ ，题目固定配置及 γ_C 。

输出：切图映射 ϕ 与序列 $\pi_0, \dots, \pi_{N-1}$ 。

1. 从 G 建立 G_c 、张量索引、拓扑序及双管道工作量。
2. 若 $N = 1$ ，返回整图单核方案；否则按式 (8.2) 计算 w_C 。
3. 按表 8-1 构造待检验集合 $\mathcal{B}$ ；置 $\mathcal{P} \leftarrow \emptyset$ 、去重集合 $\mathcal{U} \leftarrow \emptyset$ 。
4. for 固定顺序中的每个构造方案 $P \in \mathcal{B}$ do
5. 检查非 COPY 操作覆盖、子图唯一分配及依赖与核内先后的联合无环性。
6. 若检查失败或方案摘要已在 $\mathcal{U}$ 中，记录原因并跳过；否则加入 $\mathcal{U}$ 。
7. 构造 $\mathcal{H}(P)$ ，调用算法 8-1；代理事件图无法推进时记录原因并跳过。
8. 按各核分析序列及式 (8.7) 估计 $\hat{B}_{\text{spill}}$ 。
9. 计算 J_C 与服务字节数，将方案及评价量加入 $\mathcal{P}$ 。
10. end for
11. 若 $\mathcal{P} = \emptyset$ ，令 P^* 为全部操作落在核心 0 的整图方案，其余核心为空；完成结构检查与代理评价。
12. 否则按式 (8.9) 选取 P^* 。
13. return P^* 的 node_to_subgraph 与 core_schedules；诊断信息另存。

兜底分支保留了原计算图的依赖关系，适用于普通候选均未通过代理评价的情况。本轮 500 份最终方案未使用这一分支。每个图的候选与事件状态在调用时重新建立，初始 FIFO 队列为空；求解进程无需官方评估器，也无需上一轮方案作为初值。

8.5 全量配对结果与缓存收益

正式实验覆盖 100 个计算图。对每图的 2-5 核分别生成方案，单核采用固定整图方案；所有方案先冻结，再分别在无 L2 和只读 L2 配置下执行官方验收。共得到 500 组配对结果、1000 次成功评估。评价指标按第四章约定统一统计，开发与独立输入检验的关系见第九章。

对同一方案，分别记官方完工时间为 $T_{i,N}^0$ 、 $T_{i,N}^{\text{L2}}$ 。沿用第六章的原始单核参照 $T_{i,1}$ ，定义

$$\begin{aligned} S_{i,N}^0 &= \frac{T_{i,1}}{T_{i,N}^0}, & S_{i,N}^{\text{L2}} &= \frac{T_{i,1}}{T_{i,N}^{\text{L2}}}, \\ R_{i,N} &= \frac{T_{i,N}^0}{T_{i,N}^{\text{L2}}}, & \eta_{i,N} &= 1 - \frac{T_{i,N}^{\text{L2}}}{T_{i,N}^0}. \end{aligned} \quad (8.10)$$

$R_{i,N}$ 为同方案的缓存加速比， $\eta_{i,N}$ 为耗时缩短率，均在逐图计算后取算术平均。字节命中率采用官方命中字节除以命中、未命中字节之和，不与按请求次数计算的命中率混用。单核的 L2 执行时间也可能发生变化，故第三问的单核加速比通过实际配对结果计算。

表 8-2 问题三全量配对结果，每种核数均含 100 图

核数	$\bar{S}_N^0$	$\bar{S}_N^{\text{L2}}$	$\bar{R}_N$	字节命中率/%	额外搬运/MiB
1	1.000000	1.008622	1.008622	9.20	11.842
2	2.027353	2.032270	1.002481	16.09	6.349
3	2.826701	2.849607	1.009093	22.13	7.182
4	3.540343	3.591397	1.017550	22.04	7.322
5	4.080391	4.193923	1.036819	26.45	8.637

五核平均加速比为 4.193923，高于第二问的 3.952170。逐图与第二问比较，68 图耗时下降、19 图相同、13 图上升。这一比较包含切图和读取路径两方面变化；将第三问同一方案的 L2 关闭后，五核平均加速比为 4.080391。缓存开关配对得到 $\bar{R}_5 = 1.036819$ ，平均耗时缩短率为 2.976%，两种指标分别描述加速倍数与原耗时中节省的比例。

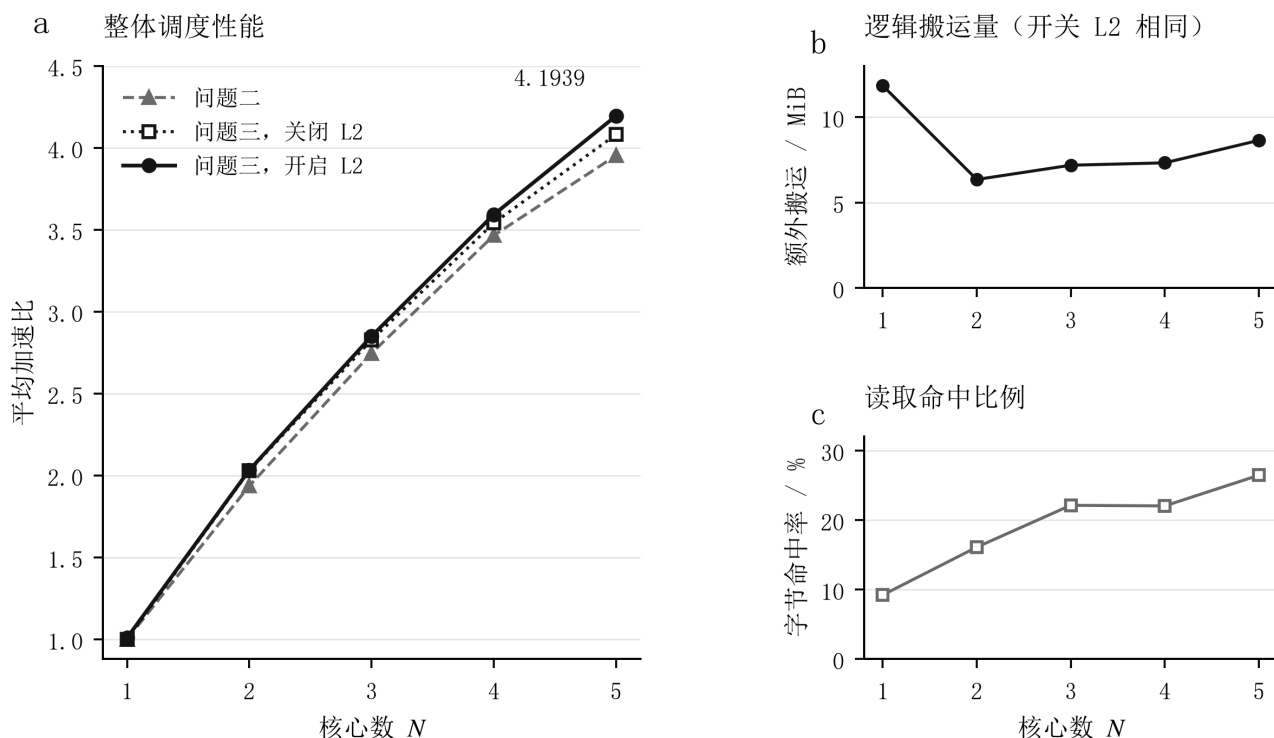


图 8-2 问题三的逐核结果。a 同时列出第二问及第三问同方案开关 L2 的平均加速比；b 为第三问的官方额外搬运量；c 为平均字节命中率。第三问每点包含 100 份实际验收结果，第二问单核点为定义值。图 b 中开关 L2 的 500 组逻辑搬运量逐项相同，因此曲线重合。

额外搬运按官方字段统计，Cache 命中只改变请求的服务路径，不直接删去这次逻辑 COPY。全部配对方案的该字段保持相同，而 DDR 实际服务压力可以降低。图 8-2 同时保留搬运量与命中率，有助于区分需要执行多少逻辑搬运、其中多少读取由 Cache 提供。

8.6 驻留时序与误差分析

五核配对中有 66 图耗时下降、33 图相同、1 图略有上升。命中率和收益之间的差异见图 8-3。case_030 的字节命中率为 71.37%，耗时从 560856 cycle 降为 560562 cycle，缩短率为 0.0524%；case_080 的命中率为 42.63%，耗时由 117286 cycle 降为 73651 cycle，缩短率为 37.20%。前者在此次 Cache 开关下的完工时间接近不变，后者具有更大的读取加速收益。

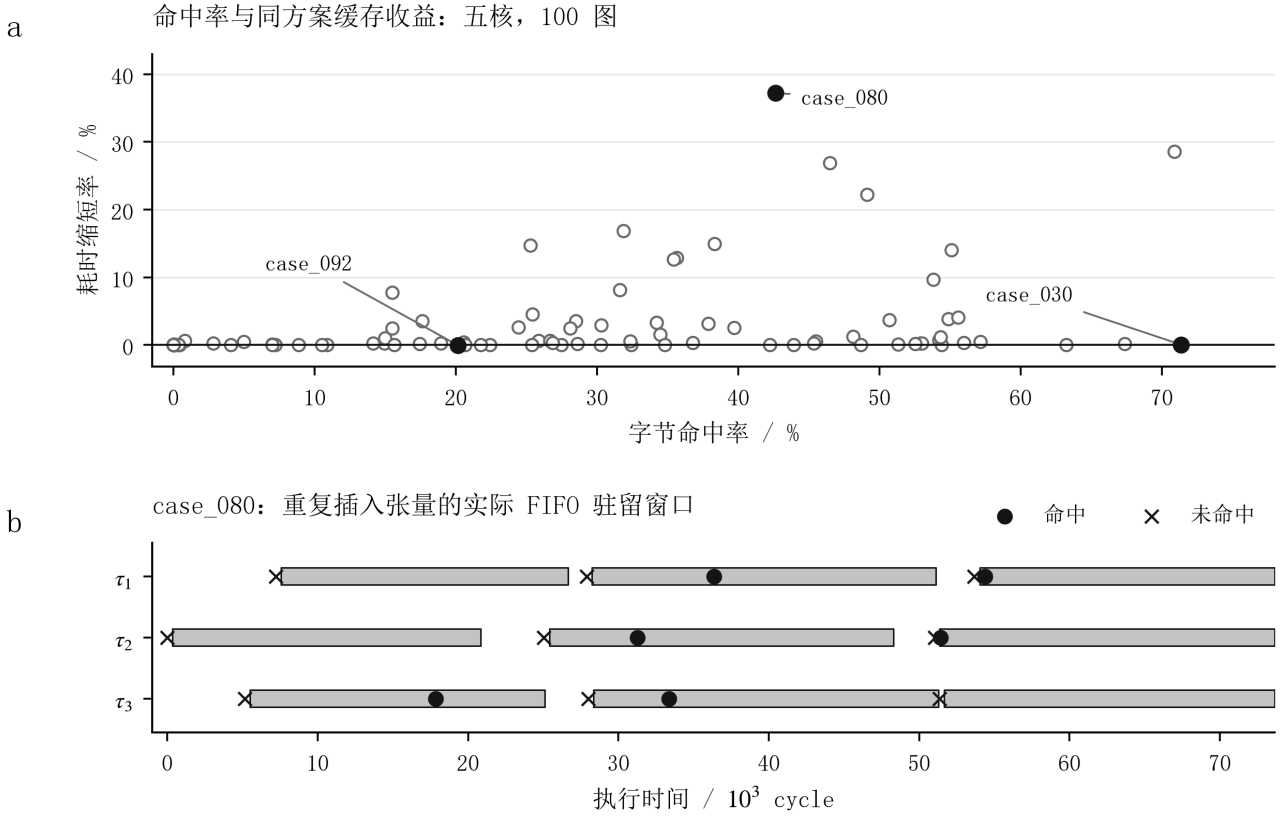


图 8-3 五核缓存效果与 FIFO 驻留窗口。a 展示全部 100 图的字节命中率与同方案耗时缩短率；b 使用缓存配对收益最大的 case_080，展示三条发生重复插入的张量轨迹。横条表示实际插入至淘汰之间的驻留，结束时仍在队列中的区间截于完工时刻；圆点和叉号分别为发射时的命中与未命中。张量按插入次数、命中次数及标识依次降序选取，原始标识和全部事件保存在图表源数据中。

驻留轨迹显示，同一逻辑张量可以经历多次插入与淘汰，命中请求也不会持续保住其在 FIFO 中的位置。读取服务加快后，操作就绪与后续请求进入带宽池的时刻随之变化；最终收益由这些变化共同决定。case_092 的完工时间从 827524 cycle 增至 827834 cycle，增加约 0.0375%，该记录在统计与图中保留。当前轻量模型还省略了官方空间复用补边和完整的换出读取序列，因而对这类细小差异的排序能力有限。

多核方案的候选数中位数为 7，最大为 10。四工作进程并发实验中，400 次多核独立求解的进程内耗时中位数为 0.600 s，最大为 10.916 s，包含输入读取、结构构造与模型评价，不含进程启动和官方验收。有限候选将每图搜索规模限制在固定范围内，事件展开提供了静态通信量之外的时序信息。对于核内换出较多或局部次序敏感的图，驻留近似仍会影响所选方案，这些误差在细粒度次序改变时可能放大，后续可针对驻留估计与事件展开分别验证。

将第二问方案记为 P_B ，第三问方案记为 P_C ，同一图上的逐级比较满足

$$\frac{T_B(P_B)}{T^{L2}(P_C)} = \frac{T_B(P_B)}{T^0(P_C)} \cdot \frac{T^0(P_C)}{T^{L2}(P_C)}. \quad (8.11)$$

左侧是最终方案相对第二问的总时间比，右侧第一项表示在场景 B 无 L2 条件下更换方案的效果，第二项表示固定第三问方案开启 L2 的效果。这一等式逐图成立，分别求平均后不能把两项均值直接相乘。第三问方案曾按 L2 条件进行选择，因此第一项也包含面向缓存构造方案所带来的排序变化，不是单一模块的独立因果效应。

五核平均加速比从第二问的 3.9522 增至第三问关闭 L2 时的 4.0804，再增至开启 L2 时的 4.1939。三条曲线说明收益既来自结构和顺序改变，也来自同一方案的读取路径变化。case_080 的缓存配对改善较大，可由驻留窗口中的有效重复读取观察；case_030 虽有较高字节命中率，完

成时间几乎不变。这些记录表明，将高命中率本身设为首要目标会忽略完成路径，本文仍以完工时间估计作为方案选择的主要依据。

第九章 综合评价与适用范围

9.1 三问的收益与计算代价

三种模型围绕同一决策接口逐步细化：问题一比较保留分量与释放内部并行的代价，问题二引入同核数据驻留，问题三展开共享读取的事件顺序。对应五核平均加速比分别为 3.8311、3.9522 和 4.1939。三者使用相同整图单核参照，平均值都先计算逐图比值再汇总；这些数字反映本题 100 图上的结果。

第一问的同场景完整分量对照用于观察扩展结构候选的收益，第二问与第一问的比较同时涉及运行机制和方案变化。第三问另行冻结同一方案切换 L2，能够直接测量该方案的缓存开关效果。因此，这三类比较回答的分别是候选扩展、跨场景方案表现和同方案硬件配置变化，分析时不能混作同一种消融实验。

算法以有限构造控制求解规模。图特征提取主要沿节点、边和张量关联表扫描；候选生成后的排序、联合图检查及驻留扫描随展开后的图规模增长。第三问进一步维护事件队列，使用更细的时间信息，400 次多核方案生成的进程内耗时中位数为 0.600 s、最大 10.916 s。该时间属于既有四工作进程实验条件，不包含官方验收和进程启动。

9.2 独立输入与等价变换检验

冷启动保证方案从当前输入重新构造；性能能否推广到其他输入，还需要额外测试。冻结后已构造串行链、独立分量、分支汇合、不规则分层、缓存压力及共享输入六类小图，并对部分图重编号和打乱数组次序。另加入 576、1152 个非 COPY 操作的两个分层图。测试图由固定种子生成，未从原 100 图抽取或拟合；正式评价在方案保存后进行。

独立测试揭示了第二问的稳定性不足。对同一个重编号后的不规则图，将原方案作等价 ID 映射得到 168070 cycle，重新求解得到 195740 cycle，耗时增加 16.46%。两项方案在同一输入上验收，排除了仅由评估器面对不同编号所产生的直接差异；候选构造或选择本身也受编号与列表顺序影响。现有记录同时改变了这两项表示，尚不能进一步分离其贡献。

表 9-1 两个新分层图的五核独立测试：两问分别按自身场景验收

操作数	问题一/cycle	问题二/cycle	问题二/问题一	生成耗时一/二/s
576	773238	2318992	2.9991	0.4040 / 0.0312
1152	1563459	4172607	2.6688	0.8077 / 0.0552

两个新图中，第二问虽然生成更快，完成时间却明显增加。其五类候选少于第一问，驻留风险又使用固定系数，二者都可能影响选择；现有对照尚未识别各因素的独立贡献。结果说明场景 B 的复用条件并不自动转化为当前算法在任意输入上的优势。后续应针对候选覆盖和评分误差分开做消融，并使用新种子确认，不能把这两个反例用于调参后继续当作独立测试。

全部题面域内的已测正常输出均完成官方模拟。审计另含直接 0p - 0p 边和非 COPY 搬运管道的越域样本，只用于程序兼容性压力检查，不计入题面域内的推广证据。问题三尚未完成同等级的新图检验，其全量结果与缓存机理验证不替代这一环节。

9.3 机理近似与提交前核对

第一问的工作量模型未显式展开动态换出，第二问的静态区间不能完全描述空间复用补边，第三问的事件图也未复制全部核内存调度。因而三问共同的改进方向是校准“构造顺序—实际驻留—请求释放”之间的误差，而非继续扩大无差别枚举。对第二问尤其需要识别因编号打破并列而改变的方案，再评估结构特征能否提供更稳定的次序。

输入契约审计还发现，冻结的问题一、二入口没有前置拒绝重复 ID、负周期、重复边和仅经 COPY 闭合的非法环；自检也未完整覆盖人为注入的负数或布尔子图编号。实际构造器在已测正常输入上生成非负整数编号，原有验收成绩不受这些注入测试改变。面向未知来源图时仍应增加独立的字段、全图 DAG 和输出契约检查，该检查无需调用性能评估器。

方案文件、参数、输入摘要、生成记录及独立验收记录一并保留。论文中的图表由相应明细重新计算，逐图时间和额外搬运表随代码提供。已测范围内，有限构造能够在较短生成时间下取得多核收益；其边界集中在输入表示敏感性、驻留近似和未见结构的候选覆盖，需要以具体反例继续改进。

参考文献

- [1] 陈怡然，王一土，神经网络加速器架构概述，中国科学：信息科学，52(4)：596 - 611，2022。DOI: 10.1360/SSI-2021-0409。
- [2] 中国研究生数学建模竞赛组委会，通用神经网络处理器下的多核调度问题及配套附件，第二十三届中国研究生数学建模竞赛 A 题，2026。
- [3] 刘正煜，张帆，祁晓峰，高彦钊，宋怡景，范旺，深度学习编译器研究综述，计算机科学，52(8)：29 - 44，2025。DOI: 10.11896/jsjx.250100062。
- [4] Graham R L, Bounds on Multiprocessing Timing Anomalies, SIAM Journal on Applied Mathematics, 17(2)：416 - 429, 1969. DOI: 10.1137/0117039。
- [5] Topcuoglu H, Hariri S, Wu M Y, Performance-effective and low-complexity task scheduling for heterogeneous computing, IEEE Transactions on Parallel and Distributed Systems, 13(3)：260 - 274, 2002. DOI: 10.1109/71.993206。
- [6] 杨紫超，吴恒，吴悦文，张文博，基于性能建模的深度学习训练任务调度综述，软件学报，36(4)：1570 - 1589，2025。DOI: 10.13328/j.cnki.jos.007202。
- [7] Chen R, Ding Z, Zheng S, 等, MAGIS: Memory Optimization via Coordinated Graph Transformation and Scheduling for DNN, Proceedings of ASPLOS 2024, Volume 3: 607 - 621, 2024。DOI: 10.1145/3620666.3651330。
- [8] Parashar A, Raina P, Shao Y S, 等, Timeloop: A Systematic Approach to DNN Accelerator Evaluation, Proceedings of ISPASS 2019: 304 - 315, 2019. DOI: 10.1109/ISPASS.2019.00042。
- [9] OpenAI, Codex 辅助工具；队伍确认的模型为 GPT 5.6 SOL、GPT 6，具体版本标识与发布日期待按使用记录核实，使用时间：2026 年 9 月。
- [10] Anthropic, Claude 人工智能工具，具体型号、版本发布日期与参与环节待队伍核实，使用时间：2026 年 9 月。

附录：结果与程序索引

配套电子表保存前两问各400行和第三问500行逐图结果。单核参照、原始方案和验收记录随三问代码包提供；附录逐图结果.tex 可并入团队提交模板。

人工智能辅助使用说明（需队伍审定）

本轮使用 Codex 辅助模型讨论、编程、数据分析和文稿整理；队伍另确认使用 Claude。工具信息见参考文献[9][10]，版本发布日期及 Claude 具体型号仍待核实。人工复核项目与代码注释模板见配套使用记录，最终文字须由队伍理解并审定。